

Name: V.Madhavreddy

Enroll No.: 2503A52698

Assignment: 8

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Requirements:

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("&quot;Abcd@123&quot;") == True
```

```
assert is_strong_password("&quot;abcd123&quot;") == False
```

```
assert is_strong_password("&quot;ABCD@1234&quot;") == True
```

Expected Output #1:

Password validation logic passing all AI-generated test cases.

Prompt:

Generate a Python program for a Password Strength Validator. Define a function `is_strong_password(password)` that returns `True` only if the password has at least 8 characters, contains at least one uppercase letter, one lowercase letter, one digit, and one special character, and does not contain spaces. Apply AI to generate at least 3 assert test cases for the function. Use these exact test cases: `assert is_strong_password("Abcd@123") == True`, `assert is_strong_password("abcd123") == False`, and `assert is_strong_password("ABCD@1234") == True`. Print a success message after all test cases pass.

Code:

```
def is_strong_password(password):  
    if len(password) < 8:  
        return False  
  
    if " " in password:  
        return False  
  
    has_upper = any(c.isupper() for c in password)  
    has_lower = any(c.islower() for c in password)  
    has_digit = any(c.isdigit() for c in password)  
    has_special = any(not c.isalnum() for c in password)
```

```
return has_upper and has_lower and has_digit and has_special
```

#exact test cases:

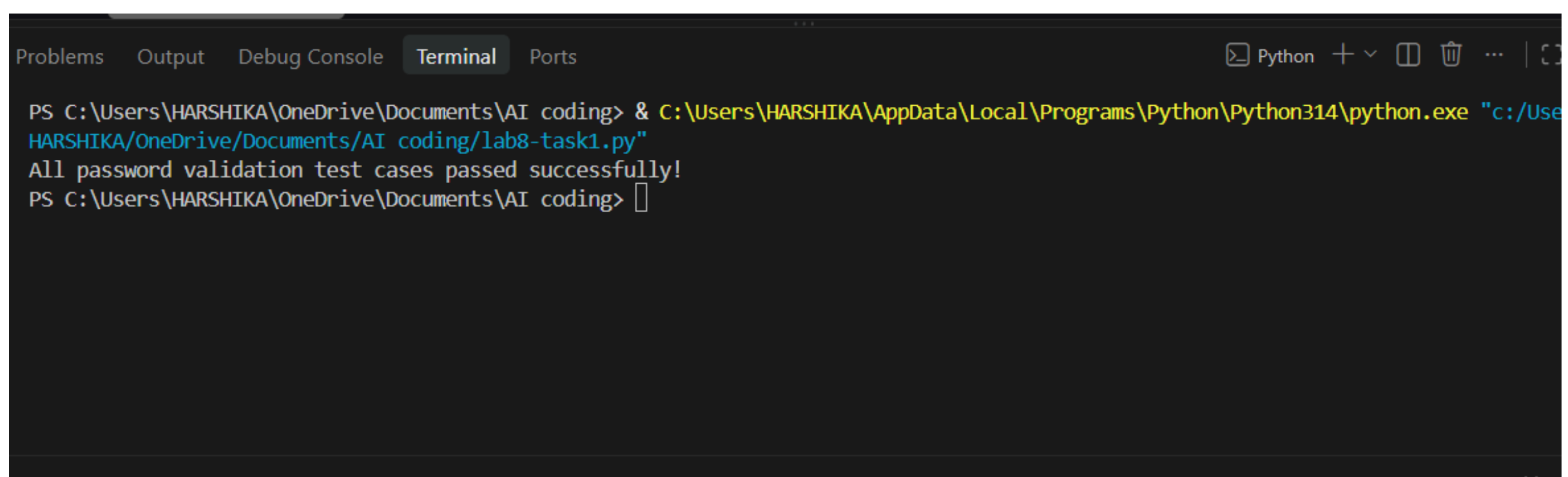
```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

```
print("All password validation test cases passed successfully!")
```

output:



```
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> & C:\Users\HARSHIKA\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/HARSHIKA/OneDrive/Documents/AI coding/lab8-task1.py"
All password validation test cases passed successfully!
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> 
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- **Task:** Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.
- **Requirements:**
 - Classify numbers as Positive, Negative, or Zero.
 - Handle invalid inputs like strings and None.
 - Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

Prompt:

```
def classify_number(n):
```

```
    if not isinstance(n, (int, float)):
```

```
        return "Invalid"
```

```
    for _ in range(1):
```

```
        if n > 0:
```

```
            return "Positive"
```

```
        elif n < 0:
```

```
            return "Negative"
```

```
        else:
```

```
            return "Zero"
```

```
# Assert test cases
```

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

```
# Boundary conditions
```

```
assert classify_number(1) == "Positive"
```

```
assert classify_number(-1) == "Negative"
```

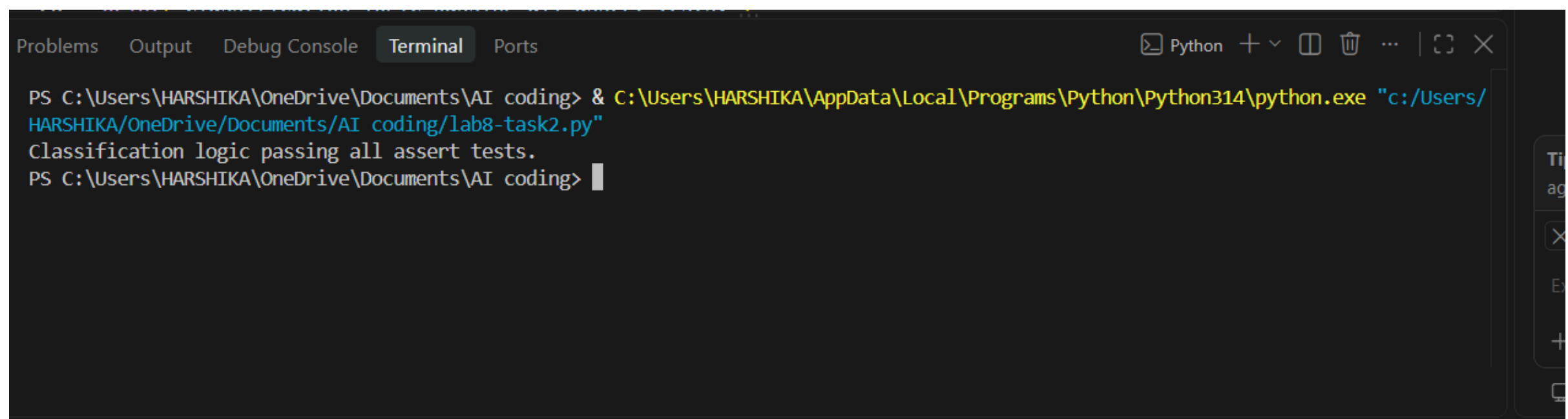
```
# Invalid inputs
```

```
assert classify_number("10") == "Invalid"
```

```
assert classify_number(None) == "Invalid"
```

```
print("Classification logic passing all assert tests.")
```

output:



```
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> & C:\Users\HARSHIKA\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/HARSHIKA/OneDrive/Documents/AI coding/lab8-task2.py"
Classification logic passing all assert tests.
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding>
```

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.
- **Requirements:**
 - Ignore case, spaces, and punctuation.
 - Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

```
assertis_anagram("listen","silent")== True
```

```
assertis_anagram("hello","world")== False
```

```
assertis_anagram("Dormitory","DirtyRoom")== True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI-generated tests.

Prompt:

Generate a Python program for an Anagram Checker using a function `is_anagram(str1, str2)`. The function should check whether two strings are anagrams while ignoring case, spaces, and punctuation. Handle edge cases such as empty strings and identical words. Generate at least 3 assert test cases using these exact examples: `assertis_anagram("listen","silent")== True`, `assertis_anagram("hello","world")== False`, and `assertis_anagram("Dormitory","DirtyRoom")== True`. Also include test cases for empty strings and identical words. Print a success message after all assert tests pass.

Code:

```
import string
```

```
defis_anagram(str1,str2):
```

```
    str1="".join(c.lower() for c in str1 if c.isalnum())
```

```
    str2="".join(c.lower() for c in str2 if c.isalnum())
```

```
    returnsorted(str1)==sorted(str2)
```

```
#Assert testcases
```

```
assertis_anagram("listen", "silent") == True
```

```
assertis_anagram("hello", "world") == False
```

```
assertis_anagram("Dormitory", "DirtyRoom") == True
```

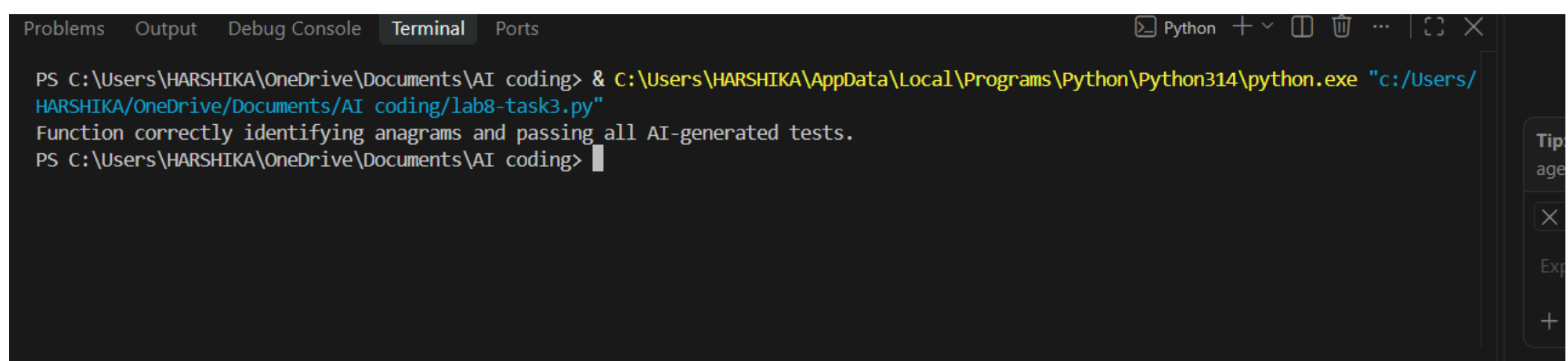
```
#Edge cases
```

```
assertis_anagram("", "") == True
```

```
assertis_anagram("hello", "hello") == True
```

```
print("Function correctly identifying anagrams and passing all AI-generated tests.")
```

output:



```
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> & C:\Users\HARSHIKA\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/HARSHIKA/OneDrive/Documents/AI coding/lab8-task3.py"
Function correctly identifying anagrams and passing all AI-generated tests.
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding>
```

Task Description #4 (Inventory Class– Apply AI to Simulate Real-World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.
- Methods:
 - add_item(name, quantity)
 - remove_item(name, quantity)
 - get_stock(name)

- Example Assert Test Cases:
- `inv = Inventory()`
- `inv.add_item("Pen", 10)`
- `assert inv.get_stock("Pen") == 10`
- `inv.remove_item("Pen", 5)`
- `assert inv.get_stock("Pen") == 5`
- `inv.add_item("Book", 3)`
- `assert inv.get_stock("Book") == 3`
- Expected Output #4:
- Fully functional class passing all assertions.

Prompt:

Generate a Python program to simulate a real-world Inventory System using an Inventory class. Implement the methods `add_item(name, quantity)`, `remove_item(name, quantity)`, and `get_stock(name)`. Use a dictionary to manage the inventory stock. Generate at least 3 assert-based test cases using these exact test cases: `inv = Inventory()`, `inv.add_item("Pen", 10)`, `assert inv.get_stock("Pen") == 10`, `inv.remove_item("Pen", 5)`, `assert inv.get_stock("Pen") == 5`, `inv.add_item("Book", 3)`, and `assert inv.get_stock("Book") == 3`. Also handle edge cases such as removing more items than available and checking an item that does not exist. Print "Fully functional class passing all assertions." after all tests pass.

Code:

```
class Inventory:

    def __init__(self):

        self.stock = {}
```

```
def add_item(self, name, quantity):  
    self.stock[name] = self.stock.get(name, 0) + quantity
```

```
def remove_item(self, name, quantity):  
    if name in self.stock and self.stock[name] >= quantity:  
        self.stock[name] -= quantity  
    else:  
        return False  
    return True
```

```
def get_stock(self, name):  
    return self.stock.get(name, 0)
```

Assert test cases

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book",3)
```

```
assert inv.get_stock("Book")==3
```

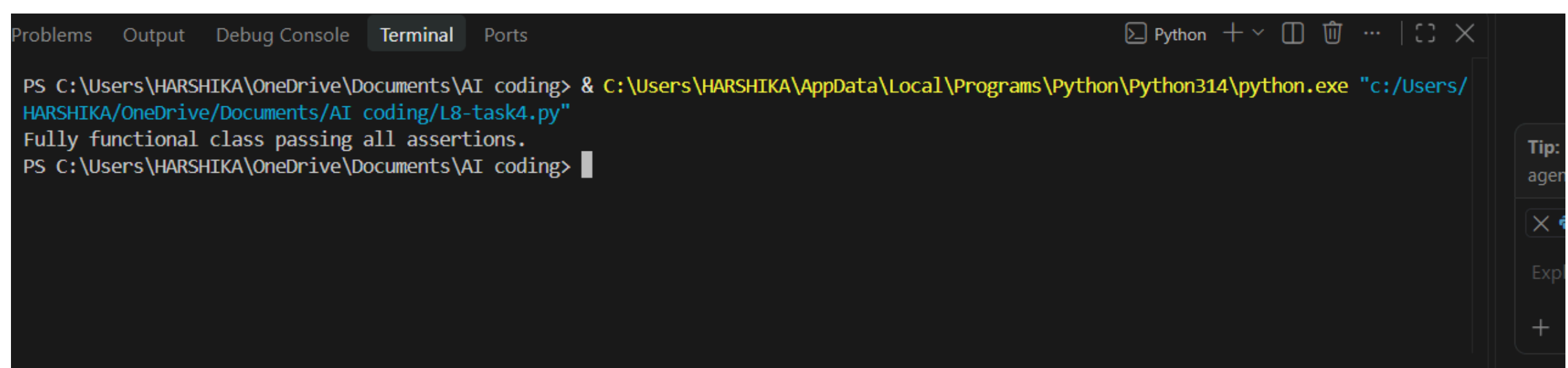
```
#Edge cases
```

```
assert inv.get_stock("Pencil")==0
```

```
assert inv.remove_item("Pen",10)==False
```

```
print("Fully functional class passing all assertions.")
```

output:



```
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> & C:\Users\HARSHIKA\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/HARSHIKA/OneDrive/Documents/AI coding/L8-task4.py"
Fully functional class passing all assertions.
PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding>
```

Task Description #5 (Date Validation&Formatting– Apply AI for Data Validation)

- **Task:** Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.
- **Requirements:**
 - Validate "MM/DD/YYYY" format.
 - Handle invalid dates.
 - Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

Prompt:

Generate a Python program for Date Validation and Formatting using a function `validate_and_format_date(date_str)`. Validate dates in "MM/DD/YYYY" format, handle invalid dates, and convert valid dates to "YYYY-MM-DD" format. Generate at least 3 assert test cases using these exact examples: `assert validate_and_format_date("10/15/2023") == "2023-10-15"`, `assert validate_and_format_date("02/30/2023") == "Invalid Date"`, and `assert validate_and_format_date("01/01/2024") == "2024-01-01"`. Also handle edge cases such as invalid formats and leap years. Print a success message after all assertions pass.

Code:

```
from datetime import datetime

def validate_and_format_date(date_str):
    try:
        date = datetime.strptime(date_str, "%m/%d/%Y")
        return date.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"
```

Assert test cases

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

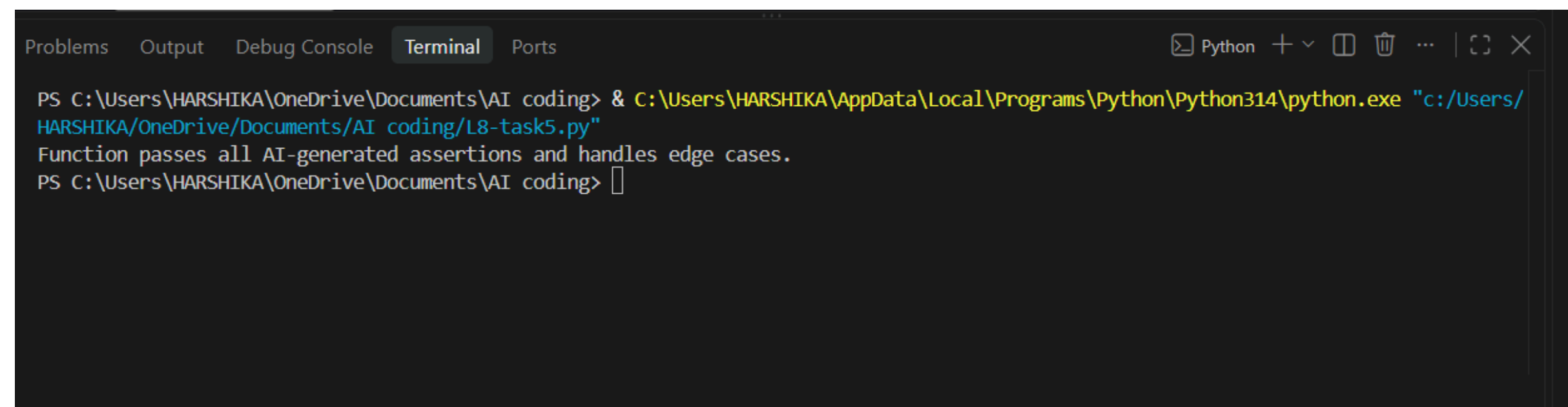
Edge cases

```
assert validate_and_format_date("02/29/2024") == "2024-02-29"
```

```
assert validate_and_format_date("2023-10-15") == "Invalid Date"
```

```
print("Function passes all AI-generated assertions and handles edge cases.")
```

output:

A screenshot of a Windows terminal window. The title bar shows 'Python' and standard window controls. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal' (selected), and 'Ports'. The command prompt shows the execution of a Python script: 'PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> & C:\Users\HARSHIKA\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/HARSHIKA/OneDrive/Documents/AI coding/L8-task5.py"'. The output of the script is 'Function passes all AI-generated assertions and handles edge cases.' followed by a new prompt line 'PS C:\Users\HARSHIKA\OneDrive\Documents\AI coding> '.